Something in the Way: Deep Learning for Crater Detection and Crater-Aware Route Planning on Mars

Enes Demirel
mdemire1@depaul.edu

November 12, 2026

Abstract

It takes approximately 4 to 22 minutes for a signal to reach Mars; therefore, Mars rovers are increasingly driven autonomously. To do this, it is important for these rovers to be able to recognize dangers such as craters without help from Earth. In this project, we study two questions: (1) which type of deep learning model performs best at detecting craters in orbital images of Mars, and (2) whether the model's mistakes matter for rovers. Using about 52,000 labeled satellite images, we compare three models: a U-Net trained from scratch, a Faster R-CNN fine-tuned from a model trained on everyday objects, and a frozen DINOv3 foundation model with only a small layer trained on top. We find that Faster R-CNN performs best with an F_1 score of 0.80, while U-Net achieves 0.57 and DINOv3 0.21. We also show that it is important to use multiple scoring methods in crater detection, as pixel-based scores can make very different models appear equally good. We then use the best detector's map to drive two simulated rovers. Rover 1 plans its full route with A*, while Rover 2 is a small neural network that learns to drive from A*'s moves, only seeing the area around itself. Both reach their goals on all 1,000 test routes, but most of the crater entries come not from missed craters, but from detected craters with slightly wrong outlines.

1 Introduction

Mars is the most habitable planet in our solar system after Earth, so it is the next step in human exploration. However, because we are not yet able to send humans to Mars; we need to rely on rovers to do the work for us. A signal from Earth to Mars, or vice versa, takes 4 to 22 minutes to reach its destination, so engineers do not have the luxury of steering a rover in real time [10]. Instead, the rover has to drive itself and for a rover to drive on its own, it is very important to be able to identify dangers on its own.

One of the most common dangers on the Martian surface is craters. One catalog alone lists more than three hundred thousand craters, and it counts only those wider than 1 km [14]. It is important for the rover to know where the craters are when planning a safe route, because if it ends up in one, it can get stuck or tip over, and there is no one to fix it. With the help of satellites, we do have orbital images that show the large areas of Mars at once, but there are far too many craters to mark by hand. This is why deep learning models are important for automatically detecting them.

However, building models to detect craters raises two questions. First, which kind of model works best? We can train a model from scratch on crater images, fine-tune a model already trained to find everyday objects [7], or use a foundation model that has learned general image features without labels. Second, do the detector's mistakes actually matter for the rover? A missed crater can lead the rover straight into it, while a false crater only makes its path a little longer. Even a good model can still miss a crater on the rover's path and cause it to crash, failing the mission, while a model with a lower score might play it safe and get the rover to its goal.

In this project, we study both questions on a public dataset of about 52,000 orbital images of the Martian surface with labeled craters [5]. First, we build crater detection networks in three different ways: a U-Net convolutional neural network (CNN) [15] trained from scratch; a Faster R-CNN [13] fine-tuned from a model that already detects everyday objects; and a DINOv3 [17] foundation model that we keep frozen, adding only a small layer on top of it. Then, we use the best model to drive two simulated rovers. Rover 1 sees the full map of detected craters and plans its whole route in advance with A* [3]. Rover 2 is a small neural network that only sees the area around itself and learns by imitating A*'s moves. Finally, we compare how often each rover reaches its goal, how often it enters a crater, and whether those craters were the ones the detector missed.

2 Related Work

Crater detection is a special case of semantic segmentation, in which every pixel in the image gets a label [8]. Silburt et al. trained a U-Net to find craters in elevation maps of the Moon, and it found 92% of the labeled craters [16]. DeLatte et al. applied a deeper U-Net to orbital images of Mars [2], and Chen et al. further improved it and trained it on THEMIS infrared images with labels from the Robbins catalog [14], the same sources our dataset is built from [1]. The success of U-Net in these papers motivated us to use a U-Net for the train-from-scratch model. However, all of these papers build a single model and judge it only with detection scores, without testing what its mistakes mean for a rover. We compare three different ways of building a model and test them on basic rover simulations.

For the rover, we follow a method that teaches a neural network to drive by copying an expert, called behavior cloning, which was first introduced with ALVINN, which learned to steer a vehicle from a human driver's actions [11, 12]. Our Rover 2 uses the same idea with A* as the expert.

3 Preliminary and Background

The crater detection problem. Each image x in our dataset is a 256×256 grid of pixels. Its mask y is a grid of the same size that marks every pixel as crater ($y_i = 1$) or not ($y_i = 0$). We use two different ways for a model to find objects in an image. A detection model, such as Faster R-CNN, draws boxes around objects, each box with a confidence score. A segmentation model, such as our U-Net, instead gives every pixel i a probability score p_i of being a crater. In both cases, whether a pixel or a box, we use the cutoff τ to determine what counts as a crater or not. A higher cutoff gives fewer but more confident detections.

Craters are among the most common features on the Martian surface. However, most craters are small, so even in heavily cratered images, most of the pixels belong to the surrounding terrain. This is called class imbalance, a well-known problem in machine learning in which one class vastly outnumbers the others. We see the same problem in our dataset, where only about 2% of the pixels in a single image belong to craters.

How we judge the detectors. Intersection over union (IoU) measures how well a predicted area A overlaps a true area B:

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \tag{1}$$

where 1 means a perfect match and 0 means no overlap. Pixel IoU compares a predicted mask to the true mask, but it counts pixels rather than craters. A model can still get a good score by finding only the large craters, which cover thousands of pixels, while missing the small ones. Therefore, we also want to know how many craters a model actually finds, and we score each crater separately.

A predicted crater counts as finding a real one if their boxes have an IoU of at least 0.5, and each real crater can be found only once. Counting the real craters that were found (TP), the predictions that matched no real crater (FP), and the real craters that were missed (FN), we compute

$$\mathrm{P} = \frac{TP}{TP + FP}, \qquad \mathrm{R} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2\,\mathrm{P}\,\mathrm{R}}{\mathrm{P} + \mathrm{R}}. \tag{2}$$

Precision (P) is the share of predictions that are real craters, and recall (R) is the share of real craters that were found. F_1 combines the two into one number, so a model cannot score well by marking everything as a crater (high recall, low precision) or by marking only the most obvious craters (high precision, low recall).

The rover simulation. Each rover navigates on a grid map, where every cell is either open or blocked by a crater. A rover moves from a start cell s to a goal cell g one step at a time. It can move into one of the 8 neighboring cells, with a cost of 1 for a straight move and $\sqrt{2}$ for a diagonal one.

Since the focus of this project is on the deep learning models, we keep the rover simulations simple and make a few assumptions. Our dataset only marks craters, so they are our only obstacles. All the maps are bird's-eye views derived from orbital imagery. The rover is modeled as a single point on the grid, where each cell corresponds to approximately 100 m on Mars. The rover always knows where it is, every move lands exactly where it was intended, and it can step in any of the eight directions without slipping or changing speed. Finally, we do not model the rover's physics or dynamics.

4 Methodology

Input Representation. Each training example is a single 256×256 grayscale image, where each pixel corresponds to approximately 0.1km on Mars. Each pixel value is scaled to the range 0 to 1. The labels are boxes around each

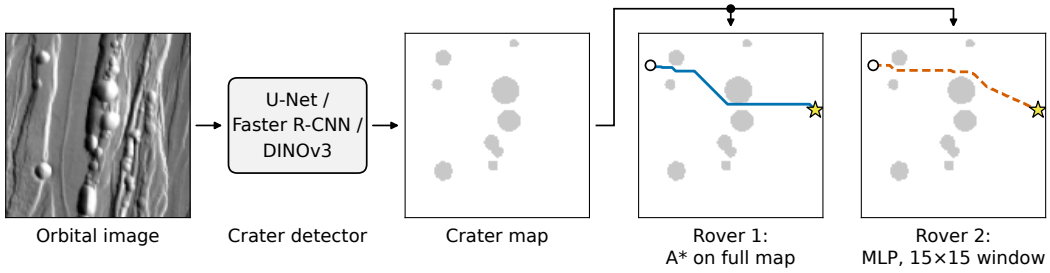


Figure 1: Overview of our pipeline, where a crater detector turns an orbital satellite image into a map of craters. Rover 1 plans its full route on this map with A*, while Rover 2 is a small network that only sees a 15×15 window around itself. Both of them start at the white circle and drive to the star.

crater, defined by their corner, width, and height. Faster R-CNN predicts boxes directly, so it uses the original boxes. However, for segmentation models, we convert each box into a filled circle centered at the same point and with a radius equal to half the box's longer side, and use these circles as a mask. Finally, both pretrained models were built for color images, so we copy the grayscale image into three identical channels. For DINOv3, we also normalize each channel with the same mean and standard deviation it was trained with.

Crater Detectors. In this project, we used three main approaches to build a model for identifying craters. A model trained from scratch starts with random weights and learns everything from our satellite images and labels. For this approach, we use U-Net [15], a neural network designed for pixel-level segmentation, and the same type of model that Silburt et al. used for lunar craters [16]. A U-Net's encoder shrinks the image three times to capture the bigger picture, and its decoder then scales it back up to full size. However, because shrinking loses fine details in the image, skip connections are used to pass these details directly from the encoder to the decoder. It gives one score per pixel and has 1.9M parameters.

Fine-tuning is a form of transfer learning that starts with a model already trained on a different task, such as object detection. Later, it continues to train on our own data. Therefore, it does not need to relearn basic image features such as shapes and edges. Here, we use a Faster R-CNN [13] with a ResNet-50 [4] backbone, trained on a large dataset of everyday objects. We replace its last layer so it predicts only two classes: crater or background. Normally, Faster R-CNN works by enlarging images to 800×800, but for the sake of training speed, our model keeps them at their original 256×256. In total, 41.1M of its parameters are trained.

Finally, to test whether a label-free model would perform well on finding craters on Mars, we use a foundation model, which is a very large model trained, usually, on millions or billions of images without labels. It learns general features of an image, such as edges, corners, and shapes, without being tied to a single task. Its weights are kept frozen, and only a small layer on top of it is trained. Here, we use the small version of DINOv3 [17], which has 21.6M parameters. This model cuts each image into patches of 16×16 and describes each patch with 384 numbers. The small layer on top, which is called the head, is a 1×1 convolution (385 parameters in total) that maps each patch to a crater score. Later, these scores are stretched back to 256×256.

For the U-Net and DINOv3, each connected group of crater pixels becomes one predicted crater, so all three models can be scored in the same way.

Training. Because of the class imbalance problem described in Section 3, with the usual binary cross-entropy loss, a model can achieve a small loss by predicting "no crater" almost everywhere. To prevent this, we use a weighted version of it that makes a mistake on a crater pixel cost more:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \Big[w\, y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big], \tag{3}$$

where N is the number of pixels and w is the penalty. We set $w = 10$ for the U-Net and the DINOv3 head. Faster R-CNN uses boxes to predict instead of pixels, so it keeps its own loss. Table 1 lists the settings for each model.

To see how much the model design matters, we also trained two simpler baselines with the same loss: an MLP (multilayer perceptron) and a basic CNN (convolutional neural network) with eight convolutions and no downsampling, unlike U-Net.

Model	Loss	Optimizer	Learning rate	Epochs	Batch size
U-Net	Eq. 3	Adam	10^{-3}, cosine	15	16
Faster R-CNN	detection losses	SGD	5×10^{-3}, cosine	15	8
DINOv3 head	Eq. 3	Adam	10^{-3}, cosine	15	64
MLP, CNN (baselines)	Eq. 3	Adam	10^{-3}, constant	10	16

Table 1: Training settings for each model. Adam [6] is the optimizer, and cosine is the cosine learning-rate schedule [9].

Model	Approach	Trained params	Precision	Recall	F_1	Pixel IoU
MLP	from scratch	0.5M	0.067	0.316	0.110	0.176
CNN	from scratch	0.2M	0.031	0.634	0.059	0.215
U-Net	from scratch	1.9M	0.481	0.703	0.571	**0.511**
Faster R-CNN	fine-tuned	41.1M	**0.795**	**0.803**	**0.799**	–
DINOv3	frozen + head	385	0.306	0.158	0.209	0.508

Table 2: Crater detection results on the test set. Pixel IoU uses each model's pixel cutoff chosen on validation: 0.8 for U-Net and DINOv3, and 0.5 for the baselines (Faster R-CNN predicts boxes, so it has none). Per-crater scores use a 0.5 cutoff for the baselines and the best validation cutoff for the main models. **Bold** means best.

All settings were chosen from a small number of options with giving most attention to defaults (details in appendix A), and all detectors were trained on an NVIDIA RTX 5090 GPU. For each model, we keep the weights and the cutoff τ with the best validation score. All models except Faster R-CNN reached their best score before their final epoch, therefore they did not need any more training with higher amount of epochs. Faster R-CNN reached its best score at its final epoch, but its validation F1 stayed between the 0.79 and 0.80 over the last epochs, so more training would likely not improve. In addition, due to model size and time constraints, we chose not to train it any further.

Rovers. Rover 1 sees the whole map, while Rover 2 sees only a small 15×15 window of it. Both use the crater map generated by Faster R-CNN, our best detector, with a 0.8 cutoff. We use an algorithm called A* [3] to find the shortest path between s and g that avoids the craters. It explores the grid by always picking the cell with the lowest cost so far plus an estimate of the distance left to the goal. Rover 1 plans the entire route before moving.

Rover 2 learns to drive using behavior cloning, a method that teaches a neural network to make decisions by learning from an expert. It works like a classic classification problem. The data comes from A* planning on the true crater map, and the network is a small MLP with 8 outputs, one for each move the rover can take. The input is what the agent sees in a given moment, and the label is the action the expert took. After training with many examples, the network predicts an action for each new input. Therefore, Rover 2 works by predicting what A* would do in each step, using only its 15×15 window and the direction to the goal. A known limitation of this method is that the expert never makes mistakes, so the training data never shows how to recover from one. After a single wrong move, the network can end up in situations it has never seen.

5 Numerical Experiments

Setup. Our Mars crater dataset comes from the 2022 GeoAI Martian Challenge [5] and consists of orbital images with crater labels from the Robbins catalog [14]. The challenge's own test images have no labels, so we keep only the 51,838 labeled images and split them randomly using the 80/10/10 rule, resulting in 41,470 for training, 5,184 for validation, and 5,184 for testing. All models used the same split. We choose the best epoch and the cutoff on the validation set, and all reported numbers are on the test set.

We test both rovers on the same routes using 1,000 different test images, selecting the start and goal from random cells that are free in both the true and detected crater maps and are at least 150 cells apart. Rover 2 is trained on the training and validation data, so the test images are new to it. A trip counts as reaching the goal if the rover arrives at it, or, for Rover 2, if it gets within 2 cells of it. We also record each true crater where the rovers enter, and count it as missed by the detector if no detected box overlaps it with an IoU of at least 0.5. Path lengths are measured with the same rule mentioned in Section 3.

	Rover 1 (A*, full map)	Rover 2 (MLP, 15×15 window)
Reached the goal	100%	100%
Routes that entered a crater	9.3%	11.2%
Craters entered	100	117
missed by the detector	32	23

Table 3: Rover results on the same 1,000 test routes.

Crater detection results. Table 2 shows the results of all models on the test set. We can see that both of the baselines perform very poorly. Both of them are not capable of seeing a whole crater at once, and both reached a pixel IoU of only about 0.2.

When we move to the main detectors, we can see that U-Net, which can see a much larger area of the image, doubles the score. Therefore, we can conclude that the model's design clearly matters.

Among the main detectors, Faster R-CNN is clearly the winner. It has an F_1 score of 0.80, with very close precision and recall scores. Therefore, we can also say that it is the most balanced. The U-Net finds most of the craters with a recall of 0.70, but fewer than half of its detections are real, with a precision of 0.48, because its masks contain many small groups of pixels that are not craters. On the other hand, DINOv3 has the opposite problem. It misses most of the craters with a recall score of 0.16. The problem is likely that each of its 16×16 patches blends craters into the surrounding ground.

We can also see the benefit of using two different ways of measuring success in our models when we looked at DINOv3 and U-Net. This is because even though they have almost the same pixel IoU (0.511 and 0.508), which means the models are about equally good per pixel, U-Net's per-crater score is almost three times better (F_1 of 0.57 against 0.21). The baselines show the same effect as well; while CNN's IoU score is better, it scores lower per crater. Therefore, we conclude that pixel IoU alone can obscure how well a model actually finds a crater, and that having an additional way to score models and a balanced way to decide the winner is much more reliable.

Rover results. Table 3 compares the two rovers on the same 1,000 test routes. We can see that both of them reach the goal on every route. Rover 1 enters a true crater in 9.3% of its routes, 100 craters in total, where 32 of them were missed by the detector and the other 68 were detected, but their circles were slightly off, so the rover drove into their edges. This shows that the precision of circle size and placement when detecting craters is as important as detecting the craters themselves. Hence, to answer our second question, missed craters do matter, but most crater entries come from imprecise crater outlines rather than misses.

While Rover 2 also reaches the goal as often, even when seeing only a 15×15 window and predicting the best move at each step, it enters more craters (11.2% of routes). Out of 117 craters it entered, only 23 were missed by the detector. This means that most were craters that Rover 2 could see but still drove into, which either happened because of the same outline mistake as discussed for Rover 1 or due to the downsides of behavior cloning discussed in Section 4. Three more example routes for each rover are shown in Appendix A.

6 Conclusion

We set out to determine whether deep learning could be useful for identifying craters on Mars's surface to help rovers plan their own routes. Our goal was to identify which kind of models detect craters the best, and whether a detector's mistakes matter for a rover. The fine-tuned Faster R-CNN performed the best (F_1 of 0.80), while U-Net trained from scratch produced many false alarms, and the frozen DINOv3 missed most craters. We also saw that having two different ways to score our models was useful, since two models with almost the same pixel IoU score did not necessarily find craters equally well. For the rovers, most crater entries came from detected craters with slightly wrong outlines, not from missed craters.

It is also important to note that our work here has some limitations. Instead of enlarging the image to 800×800 to capture finer details, we used 256×256 images to keep training fast for Faster R-CNN. DINOv3 was kept frozen, which likely hurt it on small craters. As mentioned above, our labels are boxes turned into circles; therefore, they only approximate the real crater shapes. In addition, many small craters were not labeled, some of which our detectors might have found but were counted as false detection, lowering their precision. The rover simulation is also simple, and Rover 2 only copies A*, so it never learns from its own mistakes.

Future work could include training the Faster R-CNN model on enlarged images, fine-tuning DINOv3, adding a small safety margin around detected craters, and training Rover 2 with a model that can learn from its own mistakes.

References

[1] Dong Chen, Fan Hu, P. Takis Mathiopoulos, Zhenxin Zhang, and Jiju Peethambaran. MC-UNet: Martian crater segmentation at semantic and instance levels using U-Net-based convolutional neural network. *Remote Sensing*, 15(1):266, 2023.

[2] Danielle M. DeLatte, Sarah T. Crites, Nicholas Guttenberg, Elizabeth J. Tasker, and Takehisa Yairi. Segmentation convolutional neural networks for automatic crater detection on Mars. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(9):2944–2957, 2019.

[3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[5] Chia-Yu Hsu, Wenwen Li, and Sizhe Wang. Knowledge-driven GeoAI: Integrating spatial knowledge into multi-scale deep learning for Mars crater detection. *Remote Sensing*, 13(11):2116, 2021.

[6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, 2014.

[7] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[8] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440, 2015.

[9] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. arXiv preprint arXiv:1608.03983, 2016.

[10] NASA. How to drive a mars rover. On a Mission podcast, Season 4, Episode 8.

[11] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, volume 1, pages 305–313, 1989.

[12] Dean A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.

[13] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, 2015.

[14] Stuart J. Robbins and Brian M. Hynek. A new global database of Mars impact craters $\geq$1 km: 1. Database creation, properties, and parameters. *Journal of Geophysical Research: Planets*, 117(E5), 2012.

[15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[16] Ari Silburt, Mohamad Ali-Dib, Chenchong Zhu, Alan Jackson, Diana Valencia, Yevgeni Kissin, Daniel Tamayo, and Kristen Menou. Lunar crater identification via deep learning. *Icarus*, 317:27–38, 2019.

[17] Oriane Siméoni et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025.

A Appendix

A.1 Training details

This table shows how we trained each model.

Model	Optimizer	Learning rate	Schedule	Epochs	Batch
MLP	Adam	10^{-3}	constant	10	16
CNN	Adam	10^{-3}	constant	10	16
U-Net	Adam	10^{-3}	cosine	15	16
DINOv3 head	Adam	10^{-3}	cosine	15	64
Faster R-CNN	SGD (momentum 0.9, weight decay 5×10^{-4})	5×10^{-3}	cosine	15	8
Rover 2 MLP	Adam	10^{-3}	constant	10	256

Table 4: Full training settings. Adam uses its default settings and no weight decay.

A.2 Model selection

For each model, we kept the version that did best on the validation set. The first table shows which epoch and cutoffs we kept. The second table shows how the U-Net and DINOv3 scores change as the cutoff goes up.

Model	Selected on	Best epoch	Pixel cutoff	Per-crater cutoff
MLP	validation pixel IoU	6	0.5 (fixed)	0.5 (fixed)
CNN	validation pixel IoU	7	0.5 (fixed)	0.5 (fixed)
U-Net	validation pixel IoU	10	0.8	0.9
DINOv3	validation pixel IoU	13	0.8	0.9
Faster R-CNN	validation F_1	15	–	0.8

Table 5: Chosen epoch and cutoffs for each model. The pixel cutoff gives the best validation pixel IoU; the per-crater cutoff gives the best validation F_1 (Table 6). Faster R-CNN predicts boxes, so it has no pixel cutoff.

Cutoff	U-Net	DINOv3
0.5	0.239	0.104
0.6	0.331	0.137
0.7	0.419	0.173
0.8	0.517	0.205
0.9	**0.582**	**0.214**

Table 6: Per-crater validation F_1 at each cutoff.

A.3 Validation and test results

This table puts the validation and test scores side by side. MLP and CNN only have test scores.

	Validation			Test		
Model	Precision	Recall	F_1	Precision	Recall	F_1
MLP	–	–	–	0.067	0.316	0.110
CNN	–	–	–	0.031	0.634	0.059
U-Net	0.498	0.700	0.582	0.481	0.703	0.571
DINOv3	0.311	0.163	0.214	0.306	0.158	0.209
Faster R-CNN	0.793	0.797	0.795	0.795	0.803	0.799

Table 7: Per-crater scores on the validation and test sets.

A.4 Rover details

This table lists how Rover 2 is built and trained, and the rules for its trips. The figure below shows some example routes from both rovers.

Item	Value
Network	MLP, 227 → 128 (ReLU) → 8, 30,216 parameters
Input	15 × 15 window of the detected map + direction to the goal
Labels	A* moves on the true crater map
Training data	88,252 examples from 500 training routes
Validation data	18,409 examples from 100 validation routes
Validation accuracy	75.3%
Reached the goal	within 2 cells of the goal
Step limit	3× the obstacle-free distance from start to goal
Move off the map	the rover stays in place

Table 8: Rover 2 details.

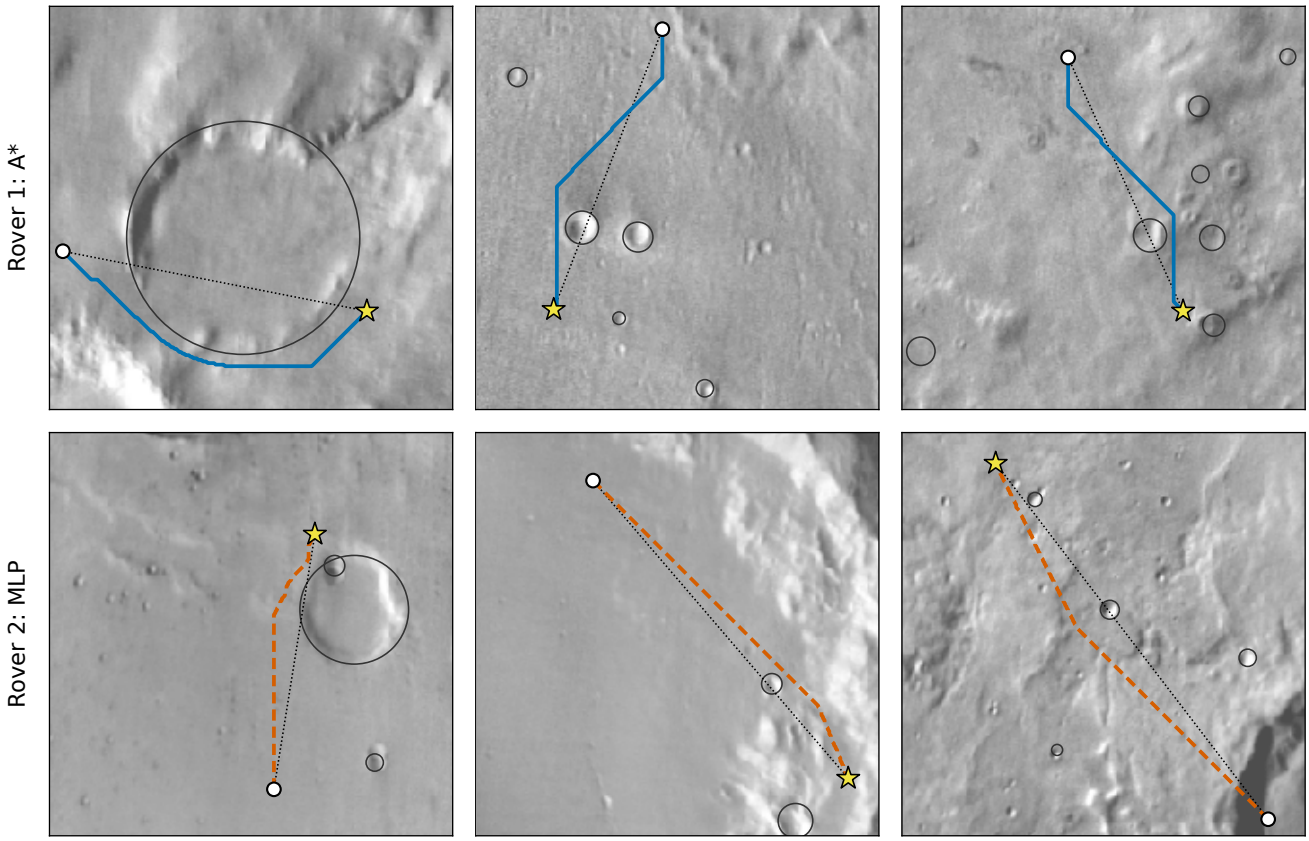


Figure 2: Example routes. Top row: Rover 1. Bottom row: Rover 2. Both start at the white circle and drive to the star.